

莫队算法

- 莫队算法是一种高效离线处理区间查询、修改问题的算法
- 它的核心思想很简单，我们去维护两个指针 l, r 来表示当前区间，通过移动指针到查询区间来得到答案，边移动边更新答案，但是如果朴素地去移动，单次移动复杂度来到 $O(n)$ ，无法接受
- 所以我们让这个指针移动尽可能少，所以我们使用根号分治，如果我们单次移动复杂度来到 $O(\sqrt{n})$ ，那么我们就得到一个复杂度为 $O(n\sqrt{n} \cdot k)$ 的算法，其中 k 是指针移动的复杂度，一般是常数或者 $\sqrt{n}$
- 那我们取块大小为 $B = \sqrt{n}$ ，以左端点所在的块的编号为第一关键字，右端点为第二关键字进行排序，那么我们就可以保证同一块内右端点移动总距离不超过 n ，每次询问时左端点移动距离不超过 B

常数优化：我们可以使奇块内的右端点按小到大排序，偶块内的大到小排序，总体呈现一个波浪型扫描

- 如果写成代码既如下

```
struct MD{
    int l,r,blk,id;
}md[N];

bool cmp(MD x,MD y){
    if(x.blk==y.blk){
        if(x.blk%2==1) return x.r<y.r;
        else return x.r>y.r;
    }
    else return x.blk<y.blk;
}

void add(int x){ }
void del(int x){ }

int main(){
    f(i,1,q){
        cin>>md[i].l>>md[i].r;
        md[i].id=i;
        md[i].blk=md[i].l/block;
    }
    sort(md+1,md+q+1,cmp);

    int tl=1,tr=0;
    int now=0;
    f(i,1,q){
        int l=md[i].l,r=md[i].r;
        while(tl>l) tl--,add(a[tl]);
        while(tl<l) del(a[tl]),tl++;
        while(tr<r) tr++,add(a[tr]);
        while(tr>r) del(a[tr]),tr--;
        ans[md[i].id]=now;
    }
}
```

```
}  
}
```

带修莫队

- 上文只实现了查询，那如果数据被修改了呢，那么我们就用到带修莫队
- 带修莫队就是加了一个时间戳，我们在查询时同时移动这个时间戳就行
- 我们先同时按照左端点所在块为第一关键字，然后若右端点在同一个块，我们以时间戳为第二关键字，否则以右端点为第二关键字
- 在此算法中，我们的块长应该取 $\sqrt[3]{n^2}$

```
const int N=3e5+10;  
int t=1;  
  
int n,m;  
struct MD{  
    int l,r,id,ti;  
}md[N];  
struct MY{  
    int id,col;  
}C[N];  
  
int block,now;  
int l,r,ql,qr,qt;  
int cnt_q=0,cnt_c=0;  
int a[N],cnt[N*10],ans[N];  
char op;  
  
bool cmp(MD x,MD y){  
    if(x.l/block==y.l/block){  
        if(x.r/block==y.r/block) return x.ti<y.ti;  
        else return x.r<y.r;  
    }  
    else return x.l<y.l;  
}  
  
void update(int l,int r,int x){  
    int id=C[x].id;  
    int &col=C[x].col;  
    if(id>=l&&id<=r){  
        now-=!cnt[a[id]];  
        now+=!cnt[col]++;  
    }  
    swap(a[id],col);  
}  
  
int main(){  
    cin>>n>>m;  
    block=pow(n,0.666);
```

```

// block=sqrt(n);
f(i,1,n) a[i]=in();
f(i,1,m){
    cin>>op;
    int x,y;
    cin>>x>>y;
    if(op=='Q'){
        ++cnt_q;
        md[cnt_q].l=x;
        md[cnt_q].r=y;
        md[cnt_q].id=cnt_q;
        md[cnt_q].ti=cnt_c;
    }
    else {
        ++cnt_c;
        C[cnt_c].id=x;
        C[cnt_c].col=y;
    }
}

sort(md+1,md+cnt_q+1,cmp);

l=0,r=0,t=0,now=0;
f(i,1,cnt_q){
    ql=md[i].l;
    qr=md[i].r;
    qt=md[i].ti;
    while(l<ql) now-=!cnt[a[l++]];
    while(l>ql) now+=!cnt[a[--l]]++;
    while(r<qr) now+=!cnt[a[++r]]++;
    while(r>qr) now-=!cnt[a[r--]];
    while(t<qt) update(ql,qr,++t);
    while(t>qt) update(ql,qr,t--);
    ans[md[i].id]=now;
}

f(i,1,cnt_q) printf("%d\n",ans[i]);
return 0;
}

```

回滚莫队

- 对于一些信息容易添加但是不易删除那怎么办，比如维护区间最值，这个信息不具有可减性
- 那么我们就去缩减区间，我们只扩展区间
- 我们先把所有区间以左端点所在的块为第一关键字，右端点为第二关键字来排序
- 对于每一个块内的询问，我们先把左端点移动到下一个块的开头，右端点移到 $l-1$ 表示空，然后我们先扩展到目标右端点，由于其有序我们可以直接扩展，完毕后我们先记录此答案，方便我们后边撤销回来；然后我们再移动左端点到目标位置得到答案

对于处于同一个块内的询问我们直接暴力处理；处理完每一个块后要清空信息

```
const int N=2e5+10;

struct MD{
    int l,r,blk,id;
    bool operator <(const MD &v) const{
        return blk!=v.blk?blk<v.blk:r<v.r;
    }
}md[N+5];
int n,m;
int block;
ll a[N],b[N];
ll cnt[N/2];
ll ans[N];
ll now=0;
ll stk[N],top;

ll add(int x,int typ){
    if(typ){
        top++;
        stk[top]=x;
    }
    cnt[x]++;
    return cnt[x]*b[x];
}

void rollbcak(){
    while(top){
        cnt[stk[top]]--;
        top--;
    }
}

void solve(){
    cin>>n>>m;
    block=sqrt(n);
    f(i,1,n){
        cin>>a[i];
        b[i]=a[i];
    }
    sort(b+1,b+n+1);
    f(i,1,n) a[i]=lower_bound(b+1,b+n+1,a[i])-b;

    int qcnt=0;
    f(i,1,m){
        int l,r;
        cin>>l>>r;
        if(l/block==r/block){
            f(j,l,r) ans[i]=max(ans[i],add(a[j],1));
            rollbcak();
            continue;
        }
    }
```

```
        qcnt++;
        md[qcnt]={l,r,l/block,i};
    }
    sort(md+1,md+qcnt+1);

    int l=0,r=0;

    f(i,1,qcnt){
        if(i==1 || md[i].blk!=md[i-1].blk){
            l=min(n+1,md[i].blk*block+block);
            r=l-1;
            now=0;
            memset(cnt,0,sizeof(cnt));
        }
        while(r<md[i].r) r++,now=max(now,add(a[r],0));
        ll tmp=now;
        while(l>md[i].l) l--,now=max(now,add(a[l],1));
        ans[md[i].id]=now;
        now=tmp;
        rollbacak();
        l=min(n+1,md[i].blk*block+block);
    }

    f(i,1,m) cout<<ans[i]<<"\n";
}
```

二次离线莫队
